



Assignment 1: Task Management Analyser

This assignment aims to help you practice Python basics including objects, primitive types, variables, operators, decision making, loops, sequences, dictionaries, functions, modules, regular expressions, exception handling, command line arguments, and file processing. Your main task in this assignment is to develop a system for analysing the logs generated by a task management system.

Overview

Many organizations use task management systems to organize their work. However, some of these systems are primitive and cannot perform the necessary analysis. One such organization has tasked you with developing a system to generate automated reports based on the output logs exported by their task management system.

The application that you will develop will process a **data.txt** files which contains the following data

```
John Smith <!jsmith>
Joe Bloggs <jbloggs>
Jane Doe <!jdoe>
Janie Roe <jroe>
Frontend <fe> -> jsmith,jbloggs
Backend <be> -> jdoe,jroe
[B1] API Development @jdoe #estimatedhours:20 #priority:high #backend #urgent
[B2] Pen-test API endpoints @jroe #estimatedhours:8 #priority:high #testing
[F1] Design Mockups @jsmith #estimatedhours:12 #priority:medium #design
[F2] User Testing @jbloggs #estimatedhours:8
[F3] Implement style guidelines @jsmith #estimatedhours:3 #priority:low #design
```

Each line will define a member, task, or team in the following ways:

- **Members:** *[Full name]* <*[username]*>: Specifies a member, their full name (which may include more than two words, such as John Leo Smith), and their username. Usernames are alphanumeric strings with no spaces while full names may contain spaces. You may assume usernames are unique.
- **Managers:** *[Full name]* <!*[username]*>: Specifies a manager similar to a member. However, their username is preceded by an "!" symbol
- **Teams:** *[Team Name]* <*[team_code]*> -> *[member1_username],[member2_username],...*: Defines a Team, its name, its unique code, and the list of members in it. You may assume each team is guaranteed to have one of its members be a manager. Team names and codes are alphanumeric strings with no spaces.
- **Tasks:** *[task_code]* *[Task Name]* @*[assigned_username]* #*[property1]:[value1]... #[tag1]...* : Defines a Task, its unique code, its name, the user it is assigned to (tasks can only be assigned to one user), and its tags and properties. Task codes, tags, property names, and property values are all alphanumeric strings with no spaces. Task names are alphanumeric and may contain spaces. Task lines will always follow the order given i.e., one assigned username then all properties then all tags.

Tags are single words preceded by a "#" symbol to categorize tasks (e.g., #urgent) while properties are name:value pairs also preceded by a "#" symbol for more precise evaluation of tasks (e.g., #estimatedhours:3). **Each task may have any number of arbitrary tags or properties.**

You must use Regular Expressions to parse this file, extract the data, and filter out any lines which do not fit the formats given above. When extracting the data you must also use Regular Expression instead of hard-coded string operations to allow for code flexibility. You may have any number of members, teams, and tasks but you may assume the lines will always follow the order of member information, team information, then task information as presented above.

Implementation Requirements

This application should receive the data file name as a command-line argument and then create the appropriate object structures. Please note that the content of the data file can be changed but their formats will be the same.

You need to name your program as **taskanalyser.py**. The file name will be passed as an argument to the program. The sample command-line execution of the program should be as follows.:

```
python taskanalyser.py data.txt
```

Please note that the name for the data file may be different. The required object class structure is presented in Figure 1.

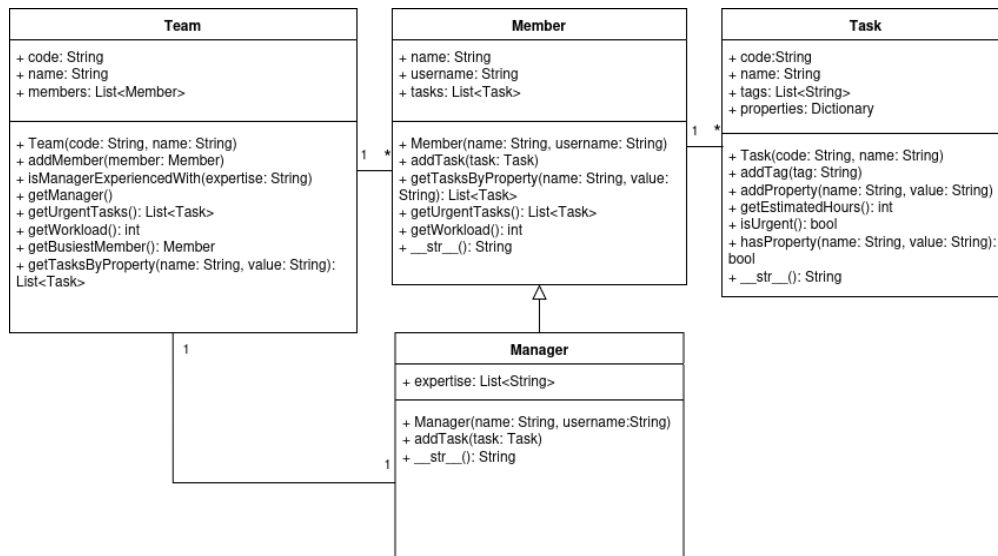


Figure 1: Class Diagram of System

You are free to add helper functions to all objects to maintain code cleanliness. The class specifications are given below.

Team Class

Attribute	Explanation
code	Unique code for the team
name	Full name of the team
members	A list of member objects of the members belonging to this team, including the manager

Method	Explanation
Team	Constructor that takes team name and code
addMember	Add a given Member object to the members list of the team
isManagerExperiencedWith	Take a particular string as expertise and returns true if the manager of the team has that expertise. (see Manager class for explanation on expertise)
getUrgentTasks	Return a list of all tasks with the tag "urgent"
getWorkload	Return a list of the total estimated hours of all tasks assigned to team members
getBusiestMember	Return the member with the greatest number of total estimated hours of tasks
getTasksByProperty	Return the list of tasks assigned to this team which contain the given property and value (e.g., getTasksByProperty("priority","high") will return all tasks which have the "priority" property and its value as "high")

Member Class

Attribute	Explanation
name	Full team member name
username	Unique team member username
tasks	List of tasks assigned to the team member

Method	Explanation
Member	Constructor that takes full name and username to create a Member object
addTask	Add a given Task object to the tasks list
getTasksByProperty	Return the list of tasks which contain the given property and value (e.g., getTasksByProperty("priority","high") will return all tasks which have the "priority" property and its value as "high")
getUrgentTasks	Return the list of all tasks assigned to this member which contain the tag "urgent"
getWorkload	Return the total estimated hours of all tasks assigned to this team member
__str__	Return a human readable string containing the member's full name, username, and the details of all tasks assigned to this team member. The string may contain newlines.

Manager Class

Inherits from Course Class

Attribute	Explanation
expertise	A list of strings representing the manager's expertise. A manager's expertise in this system is defined as all the tags of the tasks assigned directly to this manager. e.g., if a manager is assigned a task tagged "#backend #urgent" and a task tagged "#design #testing", his expertise will be backend, urgent, design, and testing. The "urgent" expertise in this case indicates that the manager has experience working on tasks under pressure.

Method	Explanation
Manager	Constructor that takes manager name and username to create a Manager object
addTask	Add a given task to the tasks list of the Manager while updating their expertise with the tags of the task.
__str__	Return a human readable string similar to Member.__str__() but with the addition of the manager's expertise

Task Class

Attribute	Explanation
code	Unique code associated with the task
name	Full task name
tags	List of tags associated with the task
properties	Dictionary of name:value property pairs associated with the task

Method	Explanation
Task	Constructor that takes task code (default="o") and name (default="Undefined") to create a Task object
addTag	Take a tag and add it to the tags list
addProperty	Take a property name and value and add them to the properties dictionary
getEstimatedHours	Return the estimated hours assigned to the task as an int. It is a policy of the organization that all tasks be assigned an estimated-hours property with the number of hours as a value.
isUrgent	Return true if the task contains the "urgent" tag, false otherwise.
hasProperty	Return true if the task contains the given property with its value as the given value, false otherwise
__str__()	Return a human-readable string describing the code, name, tags, and properties of the task. It is sufficient to format the strings in the format of the input file excluding the assigned member.

Main Program Operation

On startup, the application should take the filename as a command-line argument. The application should read the given file to populate all the objects of the system. Then, the application should show the following menu in a loop:

1. Print Managers By Expertise: The application should ask the user to enter an expertise and display the full details of all managers in the system which have that expertise
2. Print Urgent Tasks: The application should print the details of all tasks which are tagged urgent for each team. It should print the team name and all urgent tasks of the team separately for each team. If there are no urgent tasks for a team, an appropriate message should be displayed.

3. Print Team Workloads: The application should display a pie chart showing the workload in estimated hours for each team in the system. You only need to print the names of the teams on the pie chart slices. You may follow the sample code given at the end of this document. If a team has no tasks, it should be omitted from the graph.
4. Print Busiest Members: The application should print the team member with the highest workload in estimated hours for each team. It should print the name of the team and the details of the busiest member for each team. If a team has no tasks, an appropriate message should be printed.
5. Print Tasks By Property: The application should ask the user for a property name and value to search for. For each team, it prints the team name and all the tasks assigned to that team which contain that property with the specified value. If no such tasks exist for a team, an appropriate message should be printed.

Each task should be encapsulated in its own method which takes a list of teams, as well as a loadData() method which takes the data filename and returns a list of Teams with all the instantiated objects by parsing the file and extracting the data using Regular Expressions, skipping over any invalid lines. Possible exceptions should be handled gracefully.

Rules

- You need to write your program by using **Python 3.x**.
- You can **only** use all built-in functions and modules (apart from matplotlib)
- You also need to create a file called **ReadMe.txt** which contains the following items. Please note that **if you do not submit ReadMe.txt, your submission will not be evaluated**.
 - Team members
 - Which version of Python 3.x you have used
 - Which operating system you have used
 - How you have worked as a team, especially how you have divided the tasks among the team members (who was responsible for what?), how you have communicated, how you have tested the program, etc.
- You need to put all your files into a folder which is named with your student id(s) and submit the compressed version of the folder in the .zip format.
- **Only one team member** should submit the assignment.
- **Code quality, modularity, efficiency, maintainability, and appropriate comments** will be part of the grading.

Grading Policy

Grading Item	Mark (out of 100)
Validating input file lines through Regular Expressions	15
Parsing data and Creation of the object structure	20
Print Manager by Expertise	10
Print Urgent Tasks	10
Print Team Workloads	10
Print Busiest Members	10
Print Tasks By Property	10

Take command line arguments, read files and manage menu operations	10
Implement classes with correct attributes and relations	5

Printing Charts

Provided below is a sample code to print a pie chart. Please note that you need to install matplotlib beforehand.

```
import matplotlib.pyplot as plt
teams = ['Frontend', 'Backend', 'DevOps', 'DBAdmin']
workloads = [15, 28, 23, 25]
plt.pie(workloads, labels=teams)
plt.title('Team Workloads')
plt.show()
```

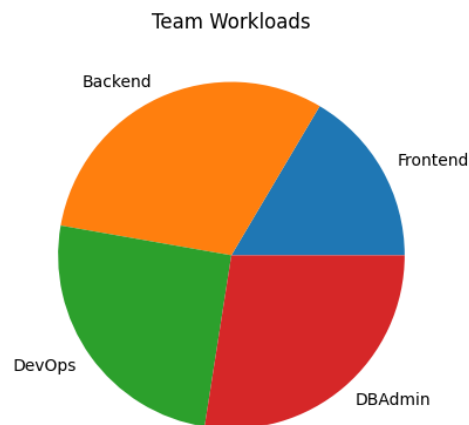


Figure 2: Sample Pie Chart

Sample Run

```
> python taskanalyser.py data.txt
```

```
Data loaded successfully.
```

```
Menu Options:
```

1. Print Manager by Expertise
2. Print Urgent Tasks
3. Print Team Workloads
4. Print Busiest Members
5. Print Tasks by Property
0. Exit

```
Enter your choice: 1
```

```
Enter expertise to search for: testing
```

```
Jane Doe <jdoe>
```

```
Expertise: testing
```

```
[B2] Pentest API endpoints #testing #estimatedhours:8 #priority:high
```

```
James Jones <jjones>
```

Expertise: database, testing
[D1] Database schema design #database #estimatedhours:15 #priority:high
[D2] Optimize queries #database #testing #estimatedhours:10

Menu Options:

1. Print Manager by Expertise
2. Print Urgent Tasks
3. Print Team Workloads
4. Print Busiest Members
5. Print Tasks by Property
0. Exit

Enter your choice: **2**

No urgent tasks for team Frontend.

Backend:

[B1] API Development #backend #urgent #estimatedhours:20 #priority:high

No urgent tasks for team DevOps.

No urgent tasks for team DBAdmin.

No urgent tasks for team MoralSupport.

Menu Options:

1. Print Manager by Expertise
2. Print Urgent Tasks
3. Print Team Workloads
4. Print Busiest Members
5. Print Tasks by Property
0. Exit

Enter your choice: **4**

Frontend:

John Smith <jsmith>

Expertise: design

[F1] Design Mockups #design #estimatedhours:12 #priority:medium

[F3] Implement style guidelines #design #estimatedhours:3 #priority:low

Backend:

Janie Roe <jroe>

[B1] API Development #backend #urgent #estimatedhours:20 #priority:high

DevOps:

Richard Roe <rroe>

Expertise: devops

[V1] Set up CI/CD pipeline #devops #estimatedhours:18 #complexity:high

DBAdmin:

James Jones <jjones>

Expertise: database, testing

[D1] Database schema design #database #estimatedhours:15 #priority:high

[D2] Optimize queries #database #testing #estimatedhours:10

No busy members in team MoralSupport.

Menu Options:

1. Print Manager by Expertise
2. Print Urgent Tasks
3. Print Team Workloads
4. Print Busiest Members
5. Print Tasks by Property
0. Exit

Enter your choice: **5**

Enter property name: **priority**

Enter property value: **high**

No such tasks in team Frontend.

Backend:

[B2] Pentest API endpoints #testing #estimatedhours:8 #priority:high

[B1] API Development #backend #urgent #estimatedhours:20 #priority:high

No such tasks in team DevOps.

DBAdmin:

[D1] Database schema design #database #estimatedhours:15 #priority:high

No such tasks in team MoralSupport.

Menu Options:

1. Print Manager by Expertise
2. Print Urgent Tasks
3. Print Team Workloads
4. Print Busiest Members
5. Print Tasks by Property
0. Exit

Enter your choice: 0

Exiting.